Exercise 1

Functions as Objects and Scipy

Task 1 *Functions as objects*

It has turned out to be a useful framework for this event to define functions that we want to minimize as classes. The definition of a function might then look like `MyFunction` in Listing 1, for example.

Listing 1: A function as a subclass of an abstract problem

```
class Problem: # Abstract class

        def value(self, x):
                raise Exception('Not implemented!')

        def gradient(self, x):
                raise Exception('Not implemented!')

        def hessian(self, x):
                raise Exception('Not implemented!')


class MyFunction(Problem): # MyFunction inherits from Problem

        def __init__(self,args):
                # Implement here e.g. the dimension of the function,
                or other attributes.
                Parameters should also be read in here.

        def value(self, x):
                # Returns the function value at the position x
                ...

        def gradient(self, x):
                # Returns the gradient at the location x
                ...

        def hessian(self, x):
                # Returns the Hessian at the location x
                ...
```

In some cases, the gradient or Hessian matrix cannot be specified analytically or at all. It could also be that they are simply not implemented yet. Therefore, it is useful to be able

to check whether the gradient or the Hessian matrix are implemented. For this, define the abstract class `Problem`, which implements the same methods as `MyFunction`, but throws an error as long as method of the abstract class is not covered by the respective method in `MyFunction`.

(a) Implement the following functions as derived classes of `problem`. In between, check that your methods serve their purpose.

- The Himmelblau-Function

$$f_1 : \mathbb{R}^2 \to \mathbb{R}, \qquad x \mapsto (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

- The quadratic function

$$f_2 : \mathbb{R}^n \to \mathbb{R}, \qquad x \mapsto \frac{1}{2} x^T Q x + p^T x + c$$

with

$$Q = T_n = \begin{pmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & \ddots & 0 & 0 \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & -1 \\ 0 & \dots & 0 & -1 & 2 \end{pmatrix}, \quad p = \text{np.ones}(n), \quad c = 0.$$

- The $\mathbb{R}^n$-Function

$$f_3 : \mathbb{R}^n \to \mathbb{R}, \quad x \mapsto \sqrt{1 + x^T Q x}$$

with $Q = T_n$.

- The $\mathbb{R}^n$-Function

$$f_4 : \mathbb{R}^n \to \mathbb{R}, \quad x \mapsto \sqrt{1 + x^T Q x} + \frac{1}{2} x^T Q x$$

with $Q = T_n$.

Task 2. *Optimization with Scipy*
Use the `scipy.optimize.minimize` library to minimize the implemented functions. See the documentation of the library.

1. Vary the starting value for the Himmelblau function. What do you observe?
2. The functions f_3 and f_4 have the trivial solution as optimum. Choose an initial value $x_0 \neq 0$ accordingly. Use 'CG', 'Newton-CG' and 'BFGS' as solution algorithm. Vary the given tolerance. What do you observe for Newton-CG compared to the other two methods?